INFORME TÉCNICO: GESTIÓN DE INCIDENCIAS (QA) Y SEGURIDAD INFORMÁTICA

- **Proyecto:** SmartGymBro
- **Grupo:** Grupo 09
- **Responsable:** Rodrigo (Auditor de QA y Especialista en Seguridad Informática)
- **Alcance:** Puntos 11 y 14 de la Guía de Operaciones (Despliegue de MantisBT, Trazabilidad con Git, Matriz de Amenazas y Plan de Respuesta ante Incidentes)
- **Fecha:** Noviembre 2026

---

1. Introducción y Objetivos

El presente informe detalla el diseño, despliegue e integración de la arquitectura de **Aseguramiento de la Calidad (QA)** y **Ciberseguridad** para el ecosistema *SmartGymBro*. El objetivo principal fue implementar un sistema centralizado de seguimiento de errores (Mantis Bug Tracker) integrado a la red de contenedores del equipo, establecer un flujo estricto de trazabilidad vinculando incidencias con commits de Git, y definir protocolos de mitigación y respuesta ante amenazas críticas.

---

2. Arquitectura y Despliegue de Mantis Bug Tracker (Punto 11)

2.1. Orquestación mediante Docker Compose

Para garantizar la portabilidad e idempotencia del entorno en las máquinas de todos los integrantes del grupo, se definió un archivo de orquestación (`infra/incidencias/compose.yaml`) compuesto por dos servicios acoplados:

- **`mantis-db` (Base de Datos Relacional):** Utiliza la imagen oficial `mysql:5.7`. Se configuró mediante variables de entorno (`MYSQL_DATABASE`, `MYSQL_USER`, `MYSQL_PASSWORD`) y se le asignó el volumen persistente `mantis_db_data` para evitar la pérdida de datos ante reinicios. Por seguridad, el puerto `3306` no se expone al exterior.
- **`mantis-web` (Aplicación Web):** Utiliza la imagen `vimagick/mantisbt:latest`. Se mapeó el puerto interno `80` hacia el puerto físico **`8089`** del host para evitar colisiones de red con servicios como Node.js o

Jenkins (puerto `8080`). Incluye la directiva `depends_on` para asegurar que el motor MySQL inicie antes que la interfaz web.
- **Interconexión de Red:** Ambos servicios operan bajo la red compartida externa **`gymbro-net`**, permitiendo que el motor de automatización (`n8n`) y las alertas de monitoreo (`Prometheus`) se comuniquen directamente de contenedor a contenedor.

2.2. Automatización de Despliegue Local (`mantis_backup.sql` y `cargar_mantis.bat`)

Dado que los volúmenes de Docker son locales a cada equipo, se exportó un respaldo lógico estructurado de la base de datos (`mysqldump`) junto con un script ejecutable en Windows (`cargar_mantis.bat`). Esto permite a cualquier miembro del equipo importar la configuración completa, usuarios, categorías e incidencias con un solo clic.

---

3. Estructura de QA y Ciclo de Vida de Incidencias

3.1. Definición de Categorías Funcionales y Roles

Dentro del proyecto **SmartGymBro** en MantisBT, se estructuraron cuatro dominios funcionales y se crearon cuentas con rol de **Desarrollador** para reflejar el flujo real del equipo:

- **Categorías:** `Backend`, `Frontend`, `Database` e `Infra / CI-CD`.
- **Usuarios del Equipo:** `administrator` (Rodrigo - QA/Admin), `aldo_front` (Aldo - Frontend), `facundo_db` (Facundo - Base de Datos) y `sonia_dev` (Sonia - Infraestructura/n8n).

3.2. Trazabilidad Estricta con Git y Flujo de Asignación

Se simuló el ciclo de vida completo de los tickets (**Nueva** → **Asignada** → **Resuelta** → **Cerrada**), documentando incidencias reales del proyecto y vinculando los identificadores únicos de Git (*Commit Hashes*) en las notas de resolución:

ID Ticket	Categoría	Resumen del Problema	Asignado a	Estado	Evidencia / Resolución Técnica
#0000001	Backend / Seg	El token JWT no expira y permite bypass de autenticación en la API.	`administrator`	Cerrada	Actualización de `jsonwebtoken` vía `npm audit fix` y validación de expiración vinculada al commit hash de Git.
#0000002	Database	Alta latencia y Timeout al cargar 6 millones de registros de sets.	`administrator`	Cerrada	Optimización de consultas SQL y creación de índices compuestos en PostgreSQL vinculada al commit hash de Git.
#0000003	Frontend	Error de renderizado al colapsar el menú de rutinas en Angular.	`aldo_front`	Asignada	Derivada al equipo de Frontend para ajuste de estilos responsive en vista móvil.
#0000004	Database	Optimización de consultas pesadas en el historial de entrenamientos.	`facundo_db`	Asignada	Derivada al equipo de Base de Datos para auditoría de planes de ejecución.
#0000005	Infra / CI-CD	Configuración de webhooks en n8n para alertas de Prometheus.	`sonia_dev`	Asignada	Derivada a automatización para apertura automática de tickets ante picos de memoria RAM.

4. Seguridad Informática y Gestión de Riesgos (Punto 14)

4.1. Auditoría Estática de Vulnerabilidades (`npm audit`)

Se ejecutó un análisis de caja blanca sobre las dependencias del servidor Backend (`package.json`), identificando vulnerabilidades de severidad alta en el manejo de sesiones (`jsonwebtoken`), las cuales fueron mitigadas mediante actualización de paquetes y endurecimiento de middlewares.

4.2. Matriz de Amenazas y Contramedidas

Se evaluaron los vectores de ataque críticos para la arquitectura de *SmartGymBro*:

ID	Amenaza	Vector de Ataque	Impacto	Contramedida Implementada
SEC-01	**Ransomware / Robo de Datos**	Exposición directa de puertos de bases de datos (`5432` Postgres / `3306` MySQL) hacia internet.	Crítico	Eliminación de mapeo de puertos externos en `compose.yaml`, aislamiento en red interna `gymbro-net` y respaldos periódicos automatizados.
SEC-02	**Denegación de Servicio (DDoS)**	Peticiones masivas automatizadas sobre endpoints de lectura intensiva (6M de registros).	Alto	Implementación de middleware `express-rate-limit` en Node.js y alertas tempranas en Prometheus/Grafana.
SEC-03	**Hijacking / Bypass JWT**	Robo de tokens de sesión sin caducidad o firmas con algoritmos débiles.	Crítico	Expiración estricta de tokens (15 min), rotación de claves secretas mediante variables de entorno y bloqueo de algoritmo `none`.
SEC-04	**Inyección SQL (SQLi)**	Manipulación de parámetros en filtros de búsqueda de rutinas.	Alto	Prohibición de concatenación de strings y uso obligatorio de consultas parametrizadas mediante ORM.
SEC-05	**Cross-Site Scripting (XSS)**	Inyección de scripts maliciosos en campos de texto libre o notas de ejercicios.	Medio	Sanitización nativa mediante `DomSanitizer` en Angular 19 y cabeceras de seguridad HTTP.

4.3. Plan de Respuesta ante Incidentes (IRP)

Ante la detección de una intrusión o caída crítica, el equipo sigue cuatro fases estandarizadas:

1. **Detección y Análisis:** Recepción de alerta automática desde Prometheus/n8n hacia MantisBT y revisión de logs de contenedores (`docker logs`).
2. **Contención:** Aislamiento inmediato del contenedor afectado desconectándolo de la red `gymbro-net` (`docker network disconnect`) sin apagarlo para preservar evidencia forense.
3. **Erradicación:** Rotación global de claves JWT y credenciales de bases de datos en los archivos `.env`, aplicando parches sobre la rama afectada mediante flujo de *Pull Request*.

4. **Recuperación:** Restauración limpia de los datos desde los respaldos verificados (`pg_dump` / `mantis_backup.sql`) y monitoreo intensivo durante las primeras 24 horas posteriores al restablecimiento.

---

5. Control de Versiones y Trabajo Colaborativo (Git Flow)

Todos los entregables técnicos (`compose.yaml`, `mantis_backup.sql`, `cargar_mantis.bat`, `matriz_amenazas.md` e `informe_incidencias.md`) fueron integrados en el repositorio oficial del grupo bajo el directorio `infra/incidencias/`. Siguiendo las normas de protección de ramas del equipo, se trabajó en ramas individuales (`feature/rodrigo-fix`, `feature/rodrigo-script`) y se integró el código hacia `develop` mediante **Pull Requests** auditados y aprobados por el líder técnico.
